SSA-Based Register Allocator for AMDGPU

Based on *Register Allocation for Programs in SSA-Form* (Hack, Grund, Goos — CC'06).

1. Motivation

General: Limitations of Iterative Graph Coloring RA

Traditional graph coloring register allocators (Chaitin [1], Briggs [2]) operate on an **iterative** spill-color-retry loop:

1. Build the interference graph
2. Attempt to k-color it
3. If coloring fails, select a node to spill
4. Materialize the spill (insert stores/reloads), rebuild the interference graph
5. Retry from step 2

This architecture has well-known deficiencies:

- **Spilling is tightly coupled to coloring.** The allocator cannot determine whether a graph is k-colorable without attempting to color it, because computing the chromatic number of a general graph is NP-complete (Chaitin [1]). Spill decisions are made reactively, based on coloring failures rather than program-level cost analysis.
- **Coalescing may increase register pressure.** Aggressive coalescing (merging copy-related nodes) can raise the chromatic number, introducing spills that would not otherwise be necessary (Hack et al. [3], Section 1, Figure 1).
- **Iteration is expensive.** The interference graph is a large data structure that must be rebuilt after each spill. For targets with small register files (high spill rates), many iterations may be required. Compilation time is difficult to bound.

Hack, Grund, and Goos [3] showed that these problems disappear when the input program is in **SSA form**. The interference graph of an SSA-form program is **chordal**, which implies:

- The chromatic number equals the maximum clique size: $\chi(G) = \omega(G)$.
- An optimal coloring can be found in $O(\omega(G) \cdot |V|)$ time via a perfect elimination order (PEO) derived from a post-order walk of the dominance tree.

- Every clique corresponds to a set of variables simultaneously live at some program point (Theorem 1). Therefore, **if the spiller reduces register pressure to at most k at every program point, the interference graph is guaranteed k-colorable** — no iteration needed.

This enables a single-pass pipeline: **Spill** → **Color** → **Coalesce** → **SSA-Destruct**, completely decoupling the four tasks.

AMDGPU-Specific: EXEC Mask Correctness for Spills and Reloads

On AMDGPU, the **EXEC mask** controls which SIMT lanes are active. When control flow diverges, the compiler modifies EXEC to mask inactive lanes. This creates a critical correctness requirement: **a spill store must capture all lanes of a VGPR, not just the currently active lanes.**

In the current AMDGPU pipeline, register allocation runs **after** SSA destruction and control flow lowering (`SILowerControlFlow`). By this point, EXEC modifications are already materialized in the MIR. The Greedy RA's `InlineSpiller` inserts spills and reloads at arbitrary program points without awareness of the EXEC state.

The primary workaround is `SIInstrInfo::isBasicBlockPrologue()` , which prevents the register allocator from inserting VGPR spills between EXEC-modifying instructions at basic block entries. It marks SGPR spills, WWM register spills, and EXEC modifications as "prologue" instructions that the allocator must not interleave with. WWM (Whole Wave Mode) wrapping provides a secondary mechanism, saving and restoring EXEC around spill sequences to ensure all lanes are captured. Both mechanisms add significant complexity to the backend and are a recurring source of subtle bugs.

An SSA-based register allocator operates **before** EXEC manipulation passes. Spills and reloads are placed while EXEC is guaranteed to be all-ones, eliminating the EXEC drift problem by construction. Neither `isBasicBlockPrologue` hacking nor WWM wrapping is needed.

AMDGPU-Specific: Register Pressure and Occupancy Control

AMDGPU occupancy (the number of concurrent wavefronts per compute unit) is **directly determined by register usage**. Fewer VGPRs used per wave = more waves = higher occupancy = better latency hiding. This makes register pressure control a first-class optimization concern, not just a correctness requirement.

The decoupled SSA-based architecture provides direct control over occupancy and enables an **iterative occupancy search**:

1. Choose a target occupancy (e.g., 4 waves → $k = 64$ VGPRs)
2. Run the spiller with RP target k
3. Evaluate spill cost: $\text{cost} = \sum(\text{spill/reload overhead})$
4. If cost is acceptable → proceed to coloring (guaranteed to succeed)
5. If cost is too high → relax to $k' = k + \Delta$, re-run the spiller only

```
target_k = max_occupancy_k

loop:
    Spill(target_k)
    cost = evaluate_spill_cost()
    if cost_acceptable(cost):
        Color()  -->  Coalesce()  -->  SSA-Destruct()
        break
    target_k += step   // relax: fewer waves, more registers
```

This is fundamentally different from the iterative loop in Greedy RA. In Greedy RA, iteration is **forced by coloring failure** — the allocator doesn't know if k colors suffice until it tries and fails. Each retry rebuilds the interference graph. In the SSA-based approach, the occupancy search loop is **driven by a cost model** — the spiller always succeeds at producing a k-bounded program, and the coloring always succeeds given that bound. The question is purely economic: is the spill overhead worth the occupancy gain? Re-running the spiller with a different k is cheap (one forward pass) and does not require rebuilding any graph.

The spiller has **freedom to use any heuristic** for spill candidate selection without worrying about coloring feasibility. In the current Greedy RA, spill decisions are constrained by the coloring failure context. In the SSA-based approach, the spiller operates independently and can optimize purely for code quality (e.g., Belady's MIN algorithm extended to whole procedures).

Bounded Compilation Time

The current Greedy RA's `selectOrSplit` runs an eviction loop where a single vreg eviction can trigger re-queuing of multiple neighbors, and each spill-retry iteration requires a full interference graph rebuild. In the worst case, the number of iterations is proportional to the number of vregs.

The SSA-based approach has bounded cost at each stage: - **Spilling**: one forward pass over the function, $O(n)$. - **Coloring**: one dominance tree walk per width class, $O(w \cdot n)$ where w is the number of distinct register widths (at most 14 on AMDGPU). - **Coalescing**: bounded by conflict graph edge additions per phi, terminates in $O(|\Phi| \cdot k^2)$.

No stage iterates. Total cost is linear in program size for a fixed register file.

Better Spill Placement

SSA dominance structure enables precise spill and reload placement via **Iterated Dominance Frontier (IDF)** analysis. Reloads are inserted only at IDF blocks of the spill point — the minimal set of locations where the spilled value needs to be made available. This produces fewer redundant reloads compared to `InlineSpiller`'s approach of splitting live ranges and inserting reloads at every use.

References

[1] G. Chaitin et al., "Register allocation via graph coloring," Journal of Computer Languages 6, 1981.

[2] P. Briggs, K. Cooper, L. Torczon, "Improvements to graph coloring register allocation," ACM TOPLAS 16(3), 1994.

[3] S. Hack, D. Grund, G. Goos, "Register Allocation for Programs in SSA-Form," CC 2006, LNCS 3923, pp. 247–262.

---

2. Background: The Paper's Key Results

The interference graph of an SSA-form program is **chordal** (Theorem 2). This yields two properties:

- **Chromatic number = max clique size**: $\chi(G) = \omega(G)$.
- **Optimal coloring via PEO**: a perfect elimination order is obtained by **post-order walk of the dominance tree**. Coloring in reverse PEO (= **pre-order** walk, i.e. dominators first) with greedy first-free gives an optimal coloring.

Combined with Theorem 1 (every clique corresponds to a program point where all its members are live), this means: **if the spiller ensures $RP \leq k$ at every point, the interference graph is k-colorable, and PEO greedy finds the coloring.**

This decouples spilling from coloring, yielding a single-pass pipeline:

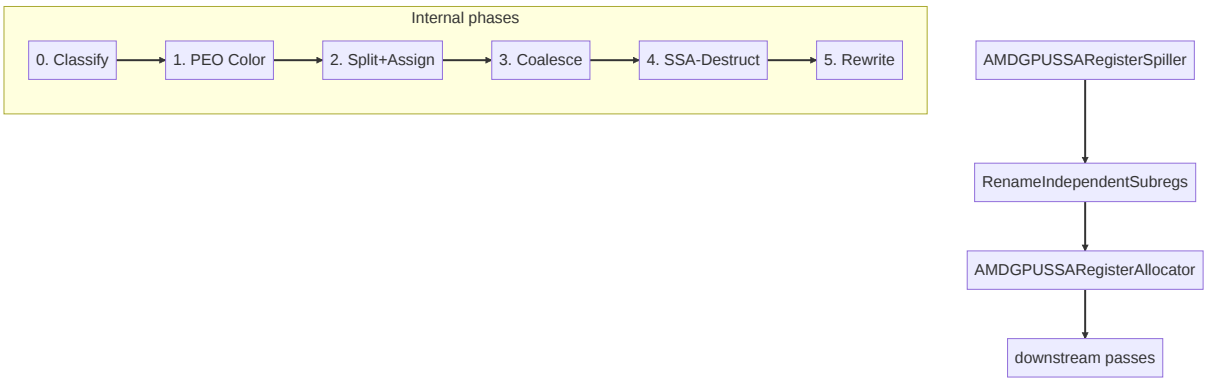
Limitations on Real Hardware

The paper assumes **uniform-width registers** (abstract interchangeable colors). On AMDGPU:

1. **Mixed-width register tuples**: virtual registers range from 32-bit to 1024-bit tuples of consecutive physical registers. A single PEO pass can produce allocations where the total distinct physregs exceeds max RP due to cross-range interference with wide vregs.
2. **Cross-range accumulation**: a narrow vreg whose live range spans multiple "epochs" of wide vregs (each using different physreg regions) must avoid all those regions globally, even though at any single point it coexists with only a subset.

The remainder of this document presents solutions: **width-descending coloring** to guarantee successful assignment, and **post-coloring LR splitting** to compact the allocation.

3. Pipeline Placement



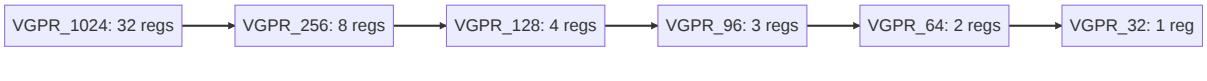
- **AMDGPUSSARegisterSpiller** (existing): guarantees $RP \leq k$ at every program point. TODO: enhance to account for tied operands in RP estimation.
- **RenameIndependentSubregs** (existing LLVM pass): splits vregs with disjoint lane live-ranges into separate smaller vregs. After this, every vreg has a single well-defined width.
- **AMDGPUSSARegisterAllocator** (new): assigns physical registers and destroys SSA form.

Hooked into AMDGPUTargetMachine.cpp, gated by `cl::opt` `-amdgpu-ssa-ra` (default off). Currently test-only via `llc -run-pass=amdgpu-ssa-register-allocator`.

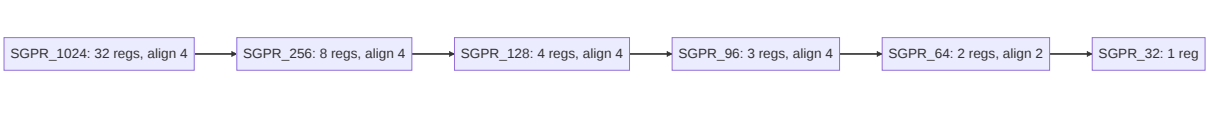
4. AMDGPU Register File Structure

Physical register tuples and alignment

VGPR file — stride = 1 for all widths (any N consecutive registers):



SGPR file — stride varies (alignment required):



Key observation from SIRegisterInfo.td: **all VGPR tuples use stride=1** (any consecutive group of N registers). SGPR tuples use stride=2 for 64-bit and stride=4 for 96-bit and wider.

Register file independence

SGPR, VGPR, and AGPR files have **non-overlapping register units**. This means: - All three files can be processed together in the same width pass - A single `BitVector OccupiedRegUnits` (indexed by `TRI->getNumRegUnits()`) tracks all files without conflict - `RegisterClassInfo::getOrder(RC)` provides per-vreg file-specific allocation order automatically

Mixed-file classes: `VS_32` / `VS_64` (SGPR+VGPR) are **non-allocatable** (`isAllocatable = 0`). `AV_32` (VGPR+AGPR) **is** allocatable but works correctly since VGPR and AGPR reg units don't overlap.

All register widths in the AMDGPU target

Width (bits)	Regs	VGPR stride	SGPR stride	Power-of-2
32	1	1	1	yes
64	2	1	2	yes
96	3	1	4	no
128	4	1	4	yes
160	5	1	4	no
192	6	1	4	no
224	7	1	4	no
256	8	1	4	yes
288	9	1	4	no
320	10	1	4	no
352	11	1	4	no
384	12	1	4	no
512	16	1	4	yes
1024	32	1	4	yes

---

5. Phase 1 — Width-Descending PEO Coloring

4.1. Algorithm Overview

Vregs are classified by width only (no per-file separation needed since register files are independent). Widths are processed in descending order. All widths use `std::set<unsigned, std::greater<unsigned>>` for sorted unique collection.

Each per-width pass walks the dominance tree in **pre-order** (= reverse PEO = dominators first). For each vreg of the current width, assigns the **smallest free physreg tuple** from `RegisterClassInfo::getOrder(RC)`.

4.2. Occupied Physreg Tracking

A single `BitVector OccupiedRegUnits` (sized `TRI->getNumRegUnits()`) tracks which physregs are currently in use. Operations:

- **Mark occupied**: `for (MCRegUnit U : TRI->regunits(PhysReg)) OccupiedRegUnits.set(U)`
- **Mark free**: `for (MCRegUnit U : TRI->regunits(PhysReg)) OccupiedRegUnits.reset(U)`
- **Check free**: iterate `RegClassInfo.getOrder(RC)`, find first physreg whose reg units are all clear

Per-point liveness: during the MDT walk, `OccupiedRegUnits` must reflect the state at the current instruction, not just a snapshot. At each instruction: - If an already-colored vreg is defined here: mark its physreg occupied - If an already-colored vreg dies here (live at current slot, dead at next): mark its physreg free - This applies to ALL already-colored vregs (wider from previous passes + same-width from earlier in current pass)

At each BB entry: seed `OccupiedRegUnits` from vregs in `ColorMap` that are live-in to the block (queried via `LiveIntervals`).

4.3. Width-Descending Advantage

Width-descending coloring produces more compact allocations than a naive single-pass approach.

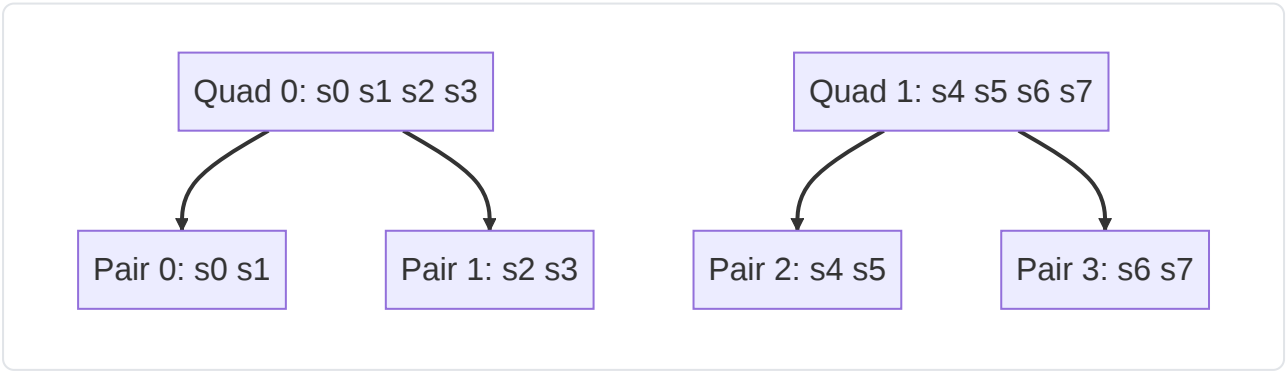
Example (5 vregs, max RP = 11, C interferes with all but B):

Approach	A(128)	B(32)	C(64)	D(32)	E(128)	Total
Width-descending	0-3	8	8-9	10	4-7	**11**
Naive (single dominance walk)	0-3	4	10-11	5	6-9	**12**

Width-descending achieves total = max RP (optimal). Naive exceeds it because B (32-bit, assigned early) occupies a slot that C (64-bit, assigned later) can't pair with.

4.4. Proof of Correctness: Aligned Register File (SGPR-style)

This proof covers architectures where register tuples must be aligned to power-of-2 boundaries (e.g., SGPRs: pairs aligned to 2, quads aligned to 4). The register file forms a strict hierarchy:



Setup. Register file of k 32-bit slots. Aligned quads: $k/4$. Aligned pairs: $k/2$. The spiller guarantees $RP \leq k$ at every program point.

Notation. At program point P with vreg v being colored: - $c = |N_w(v)|$: number of v's same-width PEO neighbors (they form a clique by the simplicial property) - q = number of distinct wider-width tuples blocking v's placement (assigned in earlier passes) - A = available tuples for v after subtracting wider blockings

Pass 1 (128-bit, aligned quads). G_{128} is chordal (induced subgraph of chordal graph). $\omega(G_{128}) \leq k/4$ (each 128-bit vreg uses 4 units, $RP \leq k$). PEO greedy with $k/4$ available quads succeeds trivially.

Pass 2 (64-bit, aligned pairs). At v's coloring point: - Pairs blocked by 128-bit assignments: $2q$ pairs (each quad blocks 2 pairs) - Available pairs: $A = k/2 - 2q$ - PEO same-width clique: c neighbors, using c distinct pairs

Because quads and pairs are hierarchically nested, 128-bit blocked pairs and 64-bit assigned pairs are **disjoint**. Greedy smallest-free fills the low-numbered (blocked) range first.

Case (a): $c \leq 2q$. All PEO neighbors fit in the blocked range. v has $A = k/2 - 2q \geq 1$ free pairs. (Holds because $q \leq (k-2)/4$, so $A \geq 1$.)

Case (b): $c > 2q$. Overflow of $(c - 2q)$ neighbors lands in v's available range. v needs: $A - (c - 2q) = k/2 - c \geq 1$. From RP at v's definition: $2(c + 1) \leq k$, so $c \leq k/2 - 1$, so $k/2 - c \geq 1$.

In both cases, v finds a free pair. Pass 3 (32-bit) follows identically.

4.5. Proof of Correctness: Unaligned Register File (VGPR-style)

On AMDGPU, all VGPR tuples have **stride = 1**: a width-W vreg needs any $W/32$ consecutive physical registers, starting at any position. This is strictly more flexible than the aligned case.

Claim. Width-descending multi-pass PEO coloring with greedy smallest-free succeeds for stride=1 register files if $RP \leq k$ at every program point.

Argument. Consider a vreg v of width W (needs $N = W/32$ consecutive slots) being colored in its pass. At v's definition point D_v:

1. **Total occupied slots** by vregs live at D_v (across all widths): $RP(D_v) \leq k$.
2. **Free slots**: $k - RP(D_v) \geq N$.
3. **Wider assignments** (from earlier passes) form contiguous blocks packed from the left.
4. **Same-width PEO neighbors** of v are packed from the left into the first available runs. They form a clique (PEO property), all live at D_v.

The remaining free space is a contiguous block at the right end of the register file. A consecutive run of N free slots always exists because the rightmost free block has size $\geq k - RP(D_v) \geq N$.

Note on cross-range constraint: v's physreg must be valid across its entire live range. If v interferes with wider vregs at different points using different physreg regions, the coloring still succeeds (a free physreg exists at each point), but the total distinct physregs used across the program may exceed max RP. The **Split+Assign** phase (section 5) addresses this.

4.6. Tied Operands

When a def operand is tied to a use operand, assign the def the same physreg as the tied use. Queried via `MI.isRegTiedToUseOperand(DefOpIdx, &UseOpIdx)` — the `MachineOperand::TiedTo` field is available in SSA form. The use is already colored (defined earlier in dominance order). The physreg transitions from the dying use to the new def without conflict.

Note: the spiller's RP estimation (`GCNRegPressure`) does NOT account for tied operands — it may overcount RP by 1 at tied instructions where the use survives. This is conservative (may cause unnecessary spills) but not incorrect. TODO: enhance spiller to handle this.

4.7. Pre-colored / Fixed Registers

Handled automatically by `RegisterClassInfo::getOrder(RC)`, which excludes all reserved physregs via `SIRegisterInfo::getReservedRegs()`: `EXEC`, `M0`, `FLAT_SCR`, stack/frame pointer, SGPRs/VGPRs past the occupancy limit, etc. ABI live-in physregs pre-occupy `OccupiedRegUnits` at function entry.

---

6. Phase 2 — Split+Assign (Post-Coloring LR Splitting)

5.1. Motivation

After Phase 1, all vregs have physregs assigned (coloring always succeeds). However, the total distinct physregs used may exceed max RP due to the cross-range accumulation problem:

A narrow vreg D whose range spans two "epochs" (epoch 1: wide vreg A alive at 0-3; epoch 2: wide vreg E alive at 4-7) must avoid both regions globally, even though at any single point D coexists with only one. D gets pushed to vgpr8+, wasting a physreg slot.

5.2. Solution: Split Wide Vregs

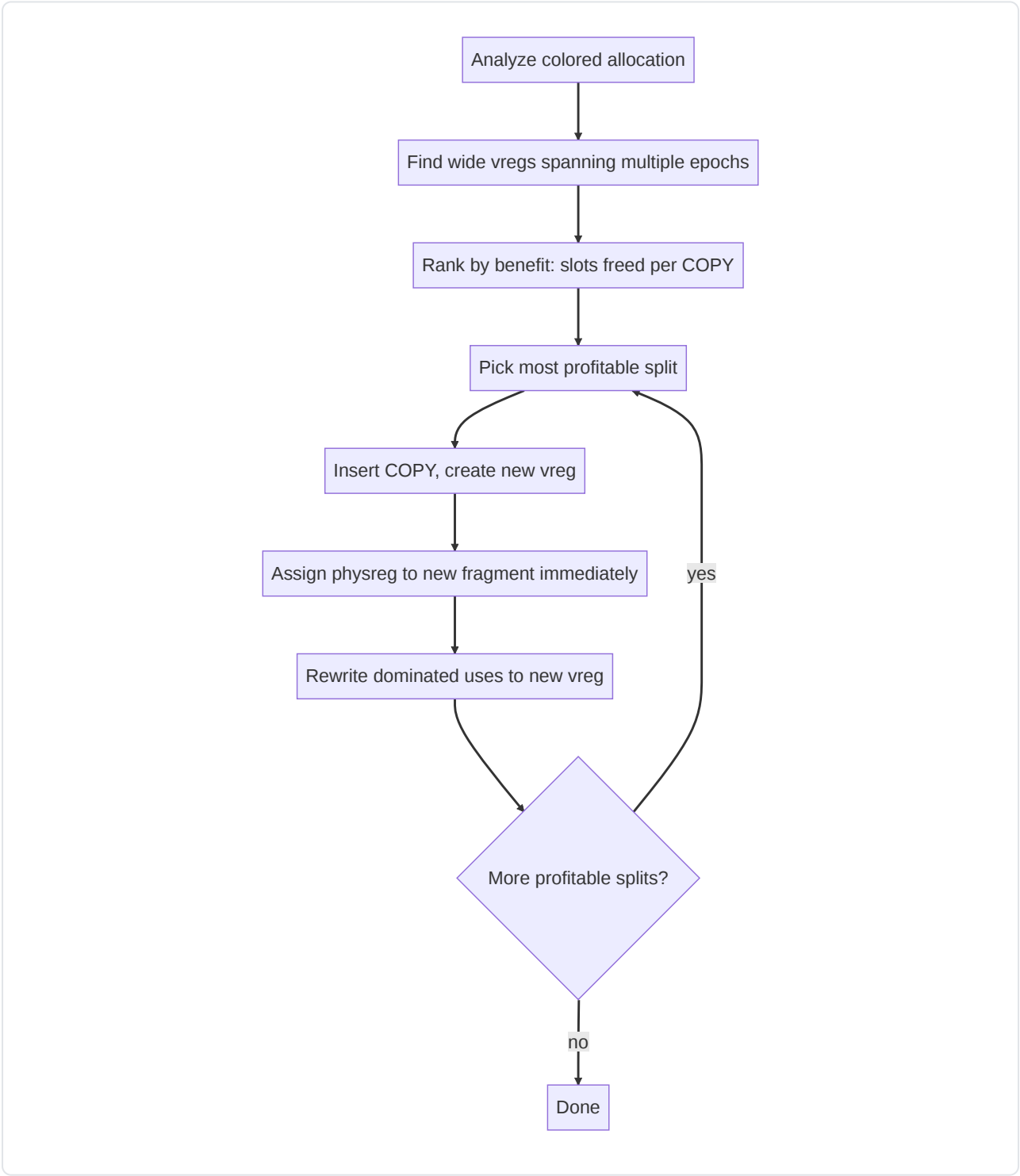
Split the **widest** vregs first — splitting a 128-bit vreg frees a quad (4 slots) for one COPY cost. Splitting a 32-bit vreg frees 1 slot for the same COPY cost.

Example: split E at the point where A dies: - Before: A→0-3, E→4-7, C→4-5, B→8, D→9. Total: 10. - After split: A→0-3, E→4-7, E'→0-3 (reuses A's space), C→4-5, B→8, D→0. Total: 9.

The COPY `E' = COPY E` does not increase RP: at the split point, A just died (freeing 0-3), E is read, E' is written to the freed space. Net RP change = 0.

5.3. Algorithm

This is a **global post-coloring optimization pass**, not a failure recovery mechanism. It analyzes the colored allocation and identifies profitable splits.



Key properties: - **Split widest first**: maximizes physreg slots freed per COPY - **Assign immediately**: at the split point, check `OccupiedRegUnits`, pick smallest free physreg for the new fragment. No separate re-coloring pass needed. - **Cost constraint**: avoid splitting inside loops. Each COPY weighted by d^2 where d = loop nesting depth. - **SSA-preserving**: the COPY creates a new vreg (V') with its own def. Uses of V dominated by the split point are rewritten to V'. V's original physreg is unchanged (its range just shortened).

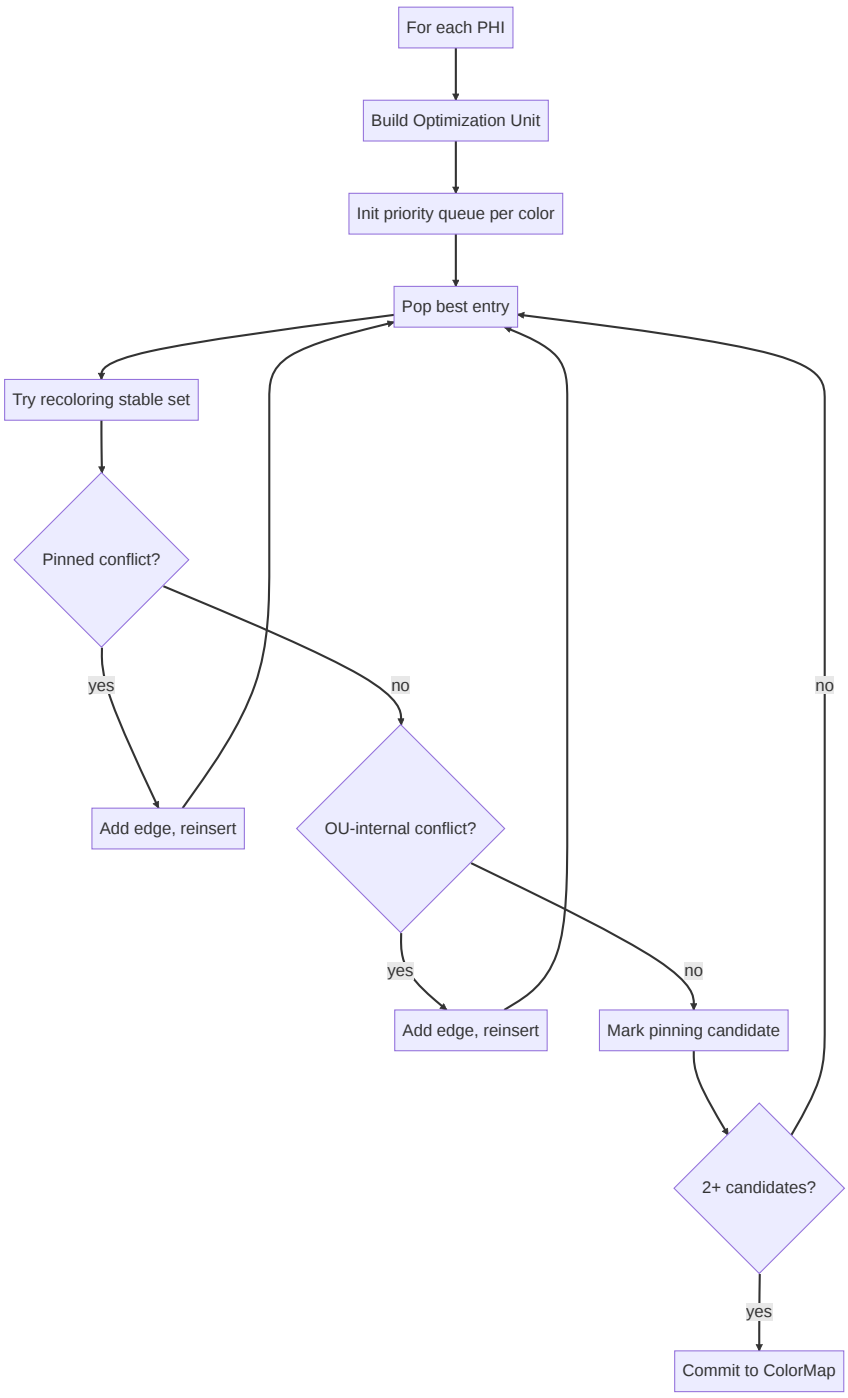
5.4. When Not to Split

Splitting is unnecessary when the coloring already achieves total distinct physregs = max RP (optimal). The pass computes "potential savings" before doing anything — if savings = 0, it's a no-op.

---

7. Phase 3 — Phi Coalesce (`AMDGPUSSAPhiCoalescer`)

Implements paper section 4.3. Mutates `ColorMap` only; does not touch MIR.

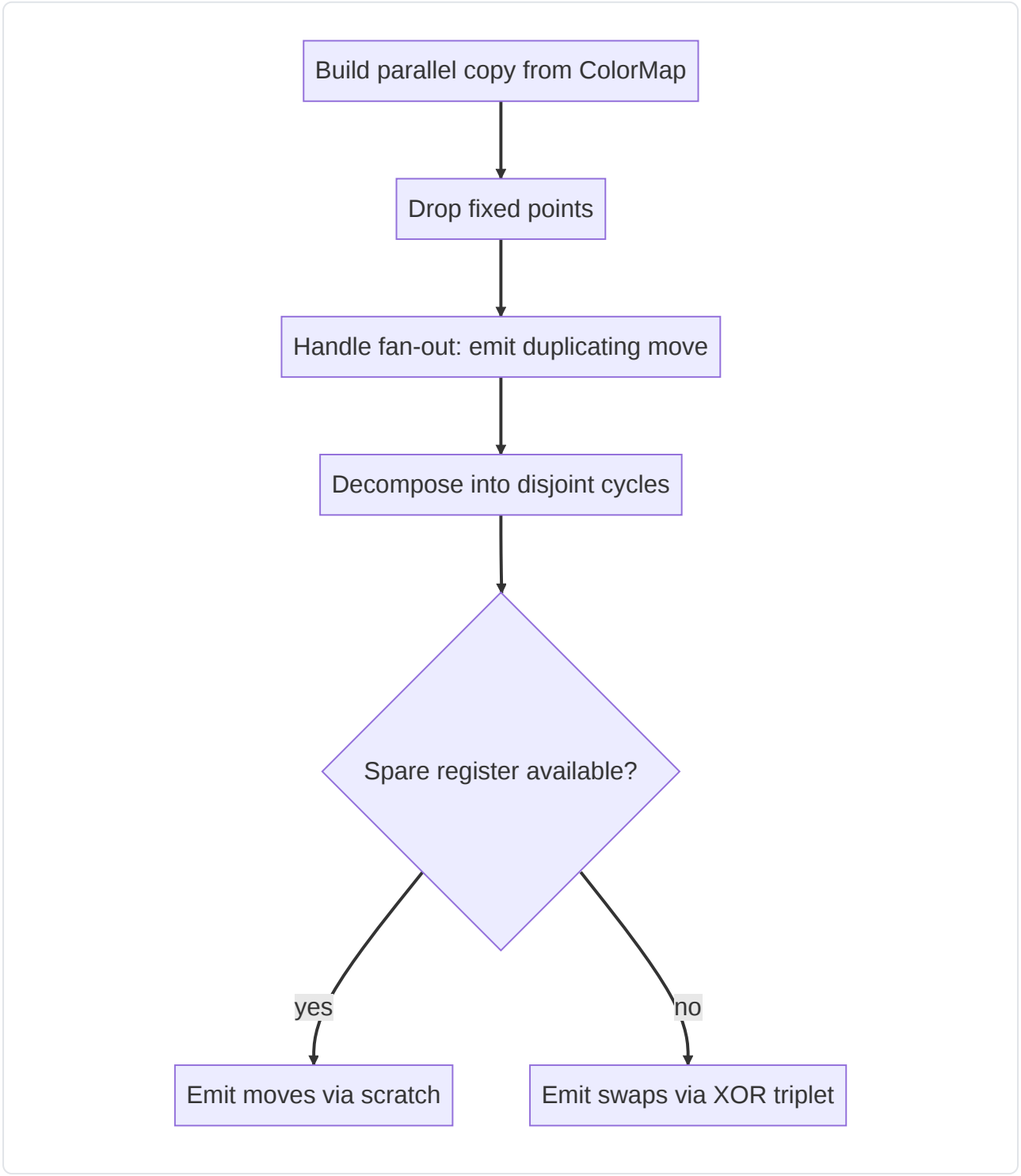


- Weights: $w_{yx} = d^2$ where d = loop nesting depth from `MachineLoopInfo`.
- Termination: each failed test adds an edge to the conflict graph. Stable set shrinks monotonically.

---

8. Phase 4 — SSA-Destruct (`AMDGPUSSADestruction`)

For each block B with phis and each incoming edge from predecessor P:



Swap lowering on AMDGPU: - VGPR: `v_xor_b32 a, a, b; v_xor_b32 b, a, b; v_xor_b32 a, a, b` - SGPR: `s_xor_b32 a, a, b; s_xor_b32 b, a, b; s_xor_b32 a, a, b` - XOR swaps work correctly under partial EXEC mask (each XOR only affects active lanes).

Edge insertion: place instructions at the end of P (before terminator) if P has a single successor, at the start of B if B has a single predecessor, otherwise split the critical edge.

---

9. Phase 5 — Operand Rewrite

Walk every `MachineOperand`, replace virtual registers with physical registers from `ColorMap`. Erase all PHI instructions (already lowered to edge copies/swaps by Phase 4).

```
for (MachineBasicBlock &BB : MF)
  for (MachineInstr &MI : make_early_inc_range(BB))
    if (MI.isPHI()) { MI.eraseFromParent(); continue; }
    for (MachineOperand &MO : MI.operands())
      if (MO.isReg() && MO.getReg().isVirtual())
        MO.setReg(ColorMap[MO.getReg()]);
```

---

10. File Layout

- `AMDGPUSSARegisterAllocator.h` / `.cpp` — `MachineFunctionPass` shell, **PEO coloring**, **LR splitting**, and **operand rewrite**.
- `AMDGPUSSAPhiCoalescer.h` / `.cpp` — paper section 4.3 heuristic. Separate class.
- `AMDGPUSSADestruction.h` / `.cpp` — phi-to-permutation lowering, cycle decomposition, swap/move emission. Separate class.

All files under `llvm/lib/Target/AMDGPU/`.

---

11. Shared Data Structures

```
class AMDGPUSSARegisterAllocator : public MachineFunctionPass {
  const SIRegisterInfo *TRI;
```

```
  const SIInstrInfo *TII;
  MachineRegisterInfo *MRI;
  MachineDominatorTree *MDT;
  LiveIntervals *LIS;
  MachineLoopInfo *MLI;
  RegisterClassInfo RegClassInfo;

  std::set<unsigned, std::greater<unsigned>> ColoringOrder;
  DenseMap<Register, MCRegister> ColorMap;
  BitVector OccupiedRegUnits;
};
```

---

12. Analyses and Pass Integration

```
void getAnalysisUsage(AnalysisUsage &AU) const override {
  AU.addRequired<LiveIntervalsWrapperPass>();
  AU.addRequired<SlotIndexesWrapperPass>();
  AU.addRequired<MachineDominatorTreeWrapperPass>();
  AU.addRequired<MachineLoopInfoWrapperPass>();
  // LIS/SlotIndexes deliberately NOT preserved.
  // Pass manager recomputes them for any downstream pass that needs them.
  MachineFunctionPass::getAnalysisUsage(AU);
}
```

---

13. Resolved Design Issues

1. **Divergent phis under EXEC** — not an issue. VGPR copies/swaps execute under correct EXEC mask; SGPR phis are uniform by construction.
2. **Pre-colored / fixed registers** — `RegisterClassInfo` excludes reserved physregs via `getReservedRegs()`. ABI live-ins pre-occupy `OccupiedRegUnits`.
3. **Tied operands** — def inherits the tied use's physreg via `MI.isRegTiedToUseOperand()`. Available in SSA form through `MachineOperand::TiedTo`. Spiller RP estimation to be enhanced (TODO).
4. **Swap lowering** — XOR triplet (`v_xor_b32` / `s_xor_b32`) when no spare register; moves via scratch when $RP < k$.

5. **Subregister coloring granularity** — `RenameIndependentSubregs` runs before our pass, splitting disjoint lane components. PHIs always define a full register. Each vreg has one width.
6. **LiveIntervals lifetime** — not preserved. Pass manager recomputes on demand.
7. **Register width fragmentation** — solved by width-descending multi-pass coloring. Proved correct for both aligned (SGPR) and unaligned (VGPR, stride=1) register files.
8. **Cross-range physreg accumulation** — coloring always succeeds but may use more distinct physregs than max RP. Post-coloring LR splitting (Phase 2) compacts the allocation by splitting wide vregs at epoch boundaries and assigning freed physregs immediately.
9. **Register file independence** — SGPR/VGPR/AGPR reg units don't overlap. No per-file separation needed in coloring order; single `OccupiedRegUnits` BitVector tracks all files.

---

14. Implementation Order

- a. **DONE**: Skeleton pass + analysis wiring + pipeline registration behind `cl::opt`.
- b. **DONE**: Pre-pass vreg width classification (`std::set<unsigned, std::greater<unsigned>>` `ColoringOrder`).
- c. Width-descending PEO coloring with per-point liveness tracking and tied operands.
- d. Operand rewrite; end-to-end test on phi-free functions.
- e. Post-coloring LR splitting (Split+Assign): wide-first, immediate physreg assignment.
- f. Naive SSA-Destruct (copies on edges, critical-edge splitting, moves via scratch).
- g. Permutation cycle decomposition + XOR swap emission.
- h. Phi coalescer (section 4.3 heuristic) + measurement of fixed-point ratio.
- i. Enhance spiller RP estimation for tied operands.

Parallel work

- Refactor AMDGPURebuildSSA to use MachineLaneSSAUpdater (worktree: `ssa-rebuilder`).